

UNIVERSIDAD DE ANTIOQUIA

Facultad de Ingeniería
Informática II – Semestre 2026-2

Implementación de Sweet Crush

mediante manipulación de bits en C++

Informe de desarrollo – Desafío I

Autores

Cristian Muñoz

Pablo Gonzalez

Medellín, septiembre de 2026

Resumen

El desafío plantea implementar un juego de tablero rectangular cuya información se conserve en una secuencia compacta de tres bits por ficha, con operaciones de eliminación, detección de combinaciones, caída de fichas, cascadas y cambios de dimensiones. La solución estudiada utiliza C++17, un único bloque dinámico de bytes para el tablero, funciones de acceso binario y módulos separados para representar el tablero, manipular los bits, resolver la lógica y mostrar el estado del juego. Se describe el razonamiento de almacenamiento, los algoritmos presentes en el repositorio, la evolución verificable de algunas funciones y los aspectos que deben comprobarse en la versión de entrega.

Palabras clave: C++, manipulación de bits, memoria dinámica, punteros, programación modular, Sweet Crush.

Índice

1. Análisis del problema y consideraciones para la solución	1
1.1. Codificación y cantidad de memoria	1
1.2. Elección de tipos de datos y restricciones	1
2. Esquema de tareas y arquitectura del sistema	2
3. Algoritmos implementados	2
3.1. Posición lógica y acceso a los tres bits	2
3.2. Creación y visualización del tablero	3
3.3. Eliminación individual y desplazamiento vertical	3
3.4. Detección horizontal y vertical	4
3.5. Caídas generales y cascadas	4
3.6. Inserción y eliminación de filas y columnas	4
3.7. Redimensionamiento y utilización de memoria	5
4. Problemas de desarrollo y aspectos por verificar	5
5. Evolución de la solución y consideraciones de implementación	6

1. Análisis del problema y consideraciones para la solución

El enunciado de Informática II solicita un tablero de dimensiones elegidas al comenzar la ejecución y formado por seis tipos de fichas. Una combinación válida contiene al menos tres fichas iguales consecutivas en una fila o una columna. Después de cada movimiento deben resolverse las coincidencias, hacer descender las fichas conservadas, agregar fichas aleatorias y repetir el proceso mientras aparezcan nuevas coincidencias. También deben admitirse inserciones y eliminaciones de filas y columnas en posiciones intermedias, con modificación física de la memoria cuando corresponda.

La restricción que determina el diseño es almacenar *exactamente tres bits por posición*, sin dedicar un byte a cada ficha y sin trasladar permanentemente el tablero a otra representación más amplia. Esto impide tratar las celdas como elementos independientes de un arreglo ordinario: una ficha puede compartir byte con otras o quedar dividida entre dos bytes. Se eligió un arreglo dinámico lineal de `unsigned char`, con lectura y escritura por máscaras y desplazamientos; las filas y columnas se interpretan de manera lógica sobre ese único arreglo.

1.1. Codificación y cantidad de memoria

La codificación visible del programa es la siguiente:

Valor	0	1	2	3	4	5
Bits	000	001	010	011	100	101
Símbolo	\$	0	X	?	#	@

Los códigos 110 y 111 no se utilizan para generar fichas en la versión examinada; las nuevas fichas se obtienen con `rand() % 6`. Para F filas y C columnas, los tamaños se calculan así:

$$N = FC, \quad B_{\text{activos}} = 3FC, \quad M = \left\lceil \frac{3FC}{8} \right\rceil, \quad S = 8M - 3FC. \quad (1)$$

Aquí M representa la cantidad mínima de bytes y S el número de bits sobrantes. Los bits sobrantes se agrupan a la **izquierda**, de modo que la secuencia válida termine en el bit menos significativo del último byte.

Ejemplo. Un tablero de 3×3 tiene 9 fichas, ocupa $9 \cdot 3 = 27$ bits y necesita $\lceil 27/8 \rceil = 4$ bytes. Los $32 - 27 = 5$ bits sobrantes quedan antes de la primera ficha: 00000 | fff fff fff fff fff fff fff fff fff. Los grupos fff representan los códigos consecutivos de las nueve fichas.

1.2. Elección de tipos de datos y restricciones

El tablero se reserva como `unsigned char*`: cada elemento guarda ocho bits de la *secuencia*, no una ficha completa. `int` se emplea para dimensiones, desplazamientos e índices; `unsigned char` permite guardar códigos de 0 a 5 y patrones binarios compactos. Las funciones reciben referencias cuando deben actualizar el puntero, las dimensiones o el número de bytes. Se mantiene una solución multiarchivo, sin `string`, `struct`, `class` ni contenedores STL. El archivo `SweetCrush.pro` declara una aplicación de consola C++17 y desactiva los módulos de Qt; se trata de un proyecto configurado mediante `qmake`, no de una interfaz gráfica.

2. Esquema de tareas y arquitectura del sistema

La solución está distribuida por responsabilidades: el archivo `.h` declara cada función y su correspondiente `.cpp` contiene su definición. El archivo `main.cpp` conecta los módulos en el flujo principal.

Módulo	Tarea principal y funciones representativas
bits	Ubicar, leer y escribir una ficha: <code>byteFicha</code> , <code>bitFicha</code> , <code>leerFicha</code> , <code>escribirFicha</code> .
tablero	Calcular y reservar memoria; generar el tablero; comprobar expansión y reducción; redimensionar; marcar fichas auxiliares.
juego	Validar entradas; eliminar y agregar fichas, filas o columnas; detectar combinaciones; efectuar caídas y cascadas; calcular puntos.
interfaz	Mostrar las fichas y sus tres bits; mostrar estadísticas y puntuación.
main	Inicializar el tablero, recibir la opción del usuario, llamar las operaciones y liberar la reserva final.

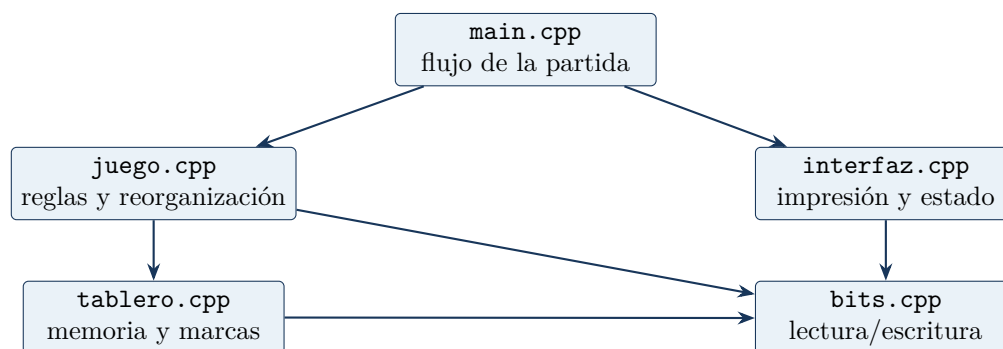


Figura 1: Dependencias funcionales principales de la implementación.

El recorrido de una operación puede resumirse como *entrada validada* → *acción sobre el tablero empaquetado* → *detección y resolución de combinaciones* → *impresión de los dos formatos y del estado*. Cuando una acción aumenta las dimensiones, primero se comprueba si la reserva existente alcanza; cuando las reduce, se revisa el umbral de utilización de memoria.

3. Algoritmos implementados

3.1. Posición lógica y acceso a los tres bits

Las coordenadas de una ficha se convierten en un índice lineal, contado desde cero:

$$i = fC + c, \quad p = 3i + S, \quad \text{byte} = \left\lfloor \frac{p}{8} \right\rfloor, \quad \text{bit} = 7 - (p \bmod 8). \quad (2)$$

La variable `bit` es la posición del primer bit de la ficha dentro del byte señalado. Si vale al menos 2, los tres bits están en un byte; si es 1 o 0, se reconstruyen a partir del byte actual y del siguiente. La función de escritura limpia primero únicamente los bits que corresponden a la ficha y luego inserta su nuevo valor.

Listing 1: Ubicación y lectura de una ficha: fragmentos del módulo `bits.cpp`.

```
1 int byteficha(int indice, int sobrantes) {  
2     int posicionBit = indice * 3 + sobrantes;  
3     return posicionBit / 8;  
4 }  
5 int bitficha(int indice, int sobrantes) {  
6     int posicionBit = indice * 3 + sobrantes;  
7     return 7 - posicionBit % 8;  
8 }  
9 // Caso contenido en un byte:  
10 return (puntero[byte] >> (bit - 2)) & 7;
```

El desplazamiento `>>` ubica el código en los tres bits menos significativos y la máscara `& 7` descarta los demás. Para escribir se utilizan `&` y `~` al borrar, y `|` al incorporar los nuevos bits, conservando las fichas vecinas. Esta abstracción permite que las demás funciones trabajen con índices de fichas, sin repetir cada vez el tratamiento de las fronteras entre bytes.

3.2. Creación y visualización del tablero

El programa solicita filas y columnas positivas, inicializa la semilla con `srand(time(0))`, calcula la cantidad de bytes y reserva `new unsigned char[bytesReservados]{}.` La función `generarTablero` recorre cada posición lógica, obtiene un valor de 0 a 5 y lo inserta en la secuencia continua. Antes del menú principal se ejecuta la resolución de posibles combinaciones iniciales, se muestra el tablero y se reinician los contadores de fichas y combinaciones detectadas.

La visualización tiene dos salidas coordinadas: `imprimirTableroBits` presenta el código binario de tres dígitos de cada ficha y `imprimirTablero` muestra su símbolo. Ambas usan `leerFicha`: ninguna requiere desempaquear el tablero completo en otro arreglo.

3.3. Eliminación individual y desplazamiento vertical

Cuando se elige una ficha, `main.cpp` valida la fila y la columna ingresadas (numeración visible desde 1). El menú convierte las coordenadas a índices internos desde cero antes de llamar a `eliminarFicha`. Desde la fila eliminada hasta la segunda, copia la ficha inmediatamente superior sobre la ubicación inferior de la misma columna. Finalmente escribe un nuevo valor aleatorio en la primera fila. Así se conserva el tamaño del tablero y se reubica únicamente la columna afectada.

Listing 2: Núcleo de la caída provocada por una eliminación individual.

```
1 for (int fila = filaEliminar; fila > 0; fila--) {  
2     int destino = fila * columnas + columnaEliminar;  
3     int origen = destino - columnas;  
4     unsigned char ficha = leerFicha(memoria, origen, sobrantes);  
5     escribirFicha(memoria, destino, sobrantes, ficha);  
6 }  
7 unsigned char nuevaFicha = rand() % 6;  
8 escribirFicha(memoria, columnaEliminar, sobrantes, nuevaFicha);
```

3.4. Detección horizontal y vertical

`detectarCombinaciones` recorre primero filas y después columnas mediante ventanas de tres fichas leídas del tablero principal. Usa las variables `anterior` y `actual` para reconocer el comienzo o continuación de una secuencia de tres o más valores iguales. Una coincidencia marca las posiciones correspondientes en un arreglo auxiliar empaquetado a **un bit por ficha**; este arreglo representa *marcas temporales*, no reemplaza la representación de tres bits por ficha del tablero original.

Bit del auxiliar	Interpretación	Acción en la caída
1	Ficha conservada	Copiar hacia abajo
0	Ficha marcada	Omitir y contabilizar

Antes de comenzar, cada byte del auxiliar se pone en 255, por lo que todas las posiciones parten como conservadas. `marcarFicha` cambia a cero el bit elegido; `fichaMarcada` recupera el estado con un desplazamiento y una máscara. El uso de una sola marca por posición permite representar intersecciones entre combinaciones horizontales y verticales sin duplicar físicamente una ficha. El cómputo de *cuántas combinaciones* se registran en esas intersecciones requiere comprobación independiente del marcado de las fichas.

3.5. Caídas generales y cascadas

Después de la detección, `realizarCaídas` procesa cada columna desde abajo hacia arriba. Mantiene un índice `lectura` y otro `escritura`; las fichas no marcadas para borrar se copian al siguiente lugar disponible desde el fondo, mientras que por cada ficha marcada se incrementa el contador de eliminadas. Las posiciones superiores sobrantes se rellenan con fichas de 0 a 5. Este procedimiento conserva el orden relativo de las fichas supervivientes dentro de cada columna.

`resolverCascadas` repite `detectarCombinaciones` mientras aumente el número de fichas eliminadas. Considera cascada adicional cada ronda de eliminación posterior a la primera. Se llama después de la inicialización y desde `tablerosYestado`, función que reúne resolución, impresión y presentación de estadísticas.

Regla de puntuación implementada: puntos = $10 \times$ fichas eliminadas en combinaciones + $20 \times$ combinaciones detectadas. El total de fichas mostrado por separado suma las eliminaciones del usuario y las eliminaciones en combinaciones.

3.6. Inserción y eliminación de filas y columnas

Las modificaciones estructurales conservan un tablero empaquetado y modifican los índices lineales directamente. Para **agregar una fila**, se desplazan hacia posiciones mayores, de atrás hacia adelante, las fichas desde el lugar de inserción; luego se generan los nuevos valores. Para **eliminar una fila**, se copian hacia posiciones posteriores las fichas anteriores a la fila removida y se reduce el número de filas.

Las columnas exigen un tratamiento por cada fila, ya que insertar o retirar una columna cambia el inicio lineal de todas las filas siguientes. `agregarColumna` y `eliminarColumna` utilizan contadores de desplazamiento y lectura/escritura de fichas para reorganizar el bloque compactado; después

actualizan el número de columnas. Las funciones reciben dimensiones por referencia para devolver su nuevo valor a `main.cpp`. El programa vuelve a buscar combinaciones después de cada modificación.

3.7. Redimensionamiento y utilización de memoria

La condición para ampliar compara los bits activos más los de una nueva fila o columna con la capacidad en bits del bloque actual. Si se necesita ampliar, `redimensionar` reserva un nuevo bloque, copia las fichas mediante `leerFicha/escibirFicha`, elimina el anterior y actualiza puntero, reserva y sobrantes. La versión examinada inicializa el bloque de destino con ceros.

La reducción física se activa solamente si se cumple

$$\underbrace{3FC}_{\text{bits en uso}} < 0,65 \underbrace{(8M)}_{\text{bits reservados}} . \quad (3)$$

La fórmula compara unidades homogéneas y corresponde al umbral exigido. El ajuste de los bits sobrantes debe mantenerse consistente con las dimensiones antes y después de cualquier operación, aun si el número de bytes reservados no cambia.

4. Problemas de desarrollo y aspectos por verificar

Cruce de fronteras entre bytes. El principal problema de representación es que las fichas no se alinean necesariamente con un byte. La solución separa los casos de una ficha contenida en un byte y de una ficha distribuida entre dos, con máscaras y desplazamientos de las porciones pertinentes.

Reducción de memoria con unidades compatibles. En una versión anterior, la condición del 65 % comparaba bits activos con una fracción del número de bytes reservados. La revisión actual incorpora el factor 8 para expresar ambos lados en bits. También se incorporó la inicialización a cero del arreglo creado por `redimensionar`, y se actualizó el parámetro de salida de memoria nueva en la rama que elimina filas.

Manipulación sin arreglo de fichas desempaquetadas. La detección necesita recordar qué posiciones van a desaparecer sin perder fichas que participan en dos orientaciones. La implementación usa un mapa temporal de un bit por posición y pospone la caída hasta terminar los recorridos horizontales y verticales.

Comprobaciones pendientes en el código consultado. Para no presentar como resueltos problemas que siguen visibles en la versión analizada, se registran estas comprobaciones antes de entregar:

- En la rama de *eliminar columna* de `main.cpp`, el cálculo de la nueva reserva aún sobrescribe la variable de capacidad antigua en vez de la de capacidad destino. Si se activa la reducción, las variables pueden quedar inconsistentes.
- Cuando una inserción cabe en los bytes actuales y no ejecuta `redimensionar`, debe ajustarse igualmente el número de bits sobrantes y, de ser necesario, reacomodarse la secuencia. Conservar el valor anterior puede desplazar la última escritura fuera del bloque.
- Conviene comprobar el contador de combinaciones cuando una ficha pertenece a coincidencias horizontales y verticales simultáneas; el mapa de fichas y el contador de combinaciones tienen criterios distintos.

- Se deben ensayar dimensiones mínimas y la eliminación de la última fila o columna, pues el menú admite llevar una dimensión a cero.

Estas observaciones provienen de la lectura del código y **no equivalen a afirmar que se ejecutaron pruebas automáticas o que todas las opciones están verificadas.**

5. Evolución de la solución y consideraciones de implementación

Etapas de representación. Se definió un único bloque lineal, la fórmula de reserva mínima y la ubicación de los bits sobrantes a la izquierda. Con ello quedaron establecidos el índice lineal y las funciones de lectura y escritura que utilizan todos los módulos posteriores.

Etapas de operaciones básicas. Sobre la representación compacta se implementaron generación, impresión binaria y simbólica, validación de entradas y eliminación individual con descenso de la columna y generación de una nueva ficha.

Etapas de modificación estructural. Se incorporaron operaciones para agregar y quitar filas y columnas, con controles de capacidad y reducción condicionada al umbral del 65 %. Esta etapa hizo necesario distinguir la cantidad de bytes reservados, la reserva calculada para un nuevo tamaño y los bits sobrantes.

Etapas de combinaciones y estado. Se añadió el marcador auxiliar de un bit por ficha, la detección en ambas direcciones, el compactado vertical, el procesamiento de cascadas y los contadores de eliminaciones, combinaciones y puntos.

Revisión del redimensionamiento. La versión más reciente examinada incorpora tres ajustes verificables: el cálculo del umbral en bits, la inicialización del nuevo bloque y la corrección del parámetro de reserva nueva en la eliminación de filas. Como consideración final, la coherencia entre tamaño, sobrantes e índices debe revisarse en todas las ramas del menú, no solo al reasignar memoria.